Chapitre 2 — Protocoles et HTTP

Notes essentielles — Master Ingénierie Informatique

Objectifs de la séance

- Définir ce qu'est un protocole
- Situer HTTP parmi les protocoles existants
- Décomposer intégralement une URL
- Maîtriser les méthodes HTTP (sûreté, idempotence)
- Décomposer une requête et une réponse HTTP
- Interpréter les codes de statut

Plan de la séance (4h)

- Qu'est-ce qu'un protocole ? (30 min)
- HTTP, le protocole du web (30 min)
- Le cycle requête-réponse et le principe stateless (30 min)
- Pause (15 min)
- Anatomie d'une requête HTTP (40 min)
- Anatomie d'une réponse HTTP (30 min)
- Comment les API exploitent HTTP (30 min)
- Exercices dirigés (15 min)

Qu'est-ce qu'un protocole ?

- Un ensemble de règles précises et partagées
- Définit comment des données sont transmises et reçues
- Leur format, leur ordre, le comportement attendu de chaque partie
- Un protocole n'a de valeur que si les deux parties l'implémentent à l'identique
- Contrairement à l'humain, un programme ne tolère pas l'approximation

Où se situe HTTP ? (modèle en couches)

- Couche Application : HTTP, FTP, SMTP, DNS
- Couche Transport : TCP, UDP
- Couche Internet : IP
- Couche Accès réseau : Ethernet, Wi-Fi
- HTTP définit uniquement la structure du message échangé
- HTTP s'appuie sur TCP pour la fiabilité (retransmission, ordre)

Pourquoi HTTP s'est-il imposé ?

- Universalité : présent dans tous les navigateurs et langages
- Simplicité relative : texte lisible, facile à déboguer
- Richesse fonctionnelle standardisée : méthodes, codes de statut, en-têtes
- Infrastructure existante : proxies, load balancers, CDN, pare-feux

HTTP, le protocole du web

- Protocole de la couche application
- Modèle requête / réponse
- Par nature sans état (stateless)
- HTTP/0.9 (1991) → HTTP/1.0 (1996) → HTTP/1.1 (1997)
- HTTP/2 (2015) : multiplexage, compression des en-têtes
- HTTP/3 (2022) : repose sur QUIC (UDP) plutôt que TCP

HTTPS

- HTTPS = HTTP transporté au-dessus d'une couche de chiffrement (TLS)
- La structure des messages reste rigoureusement identique
- Seule la confidentialité / intégrité du transport change
- Détaillé au Chapitre 7 (Sécurité)

Le cycle requête-réponse

- Le client envoie une requête HTTP (méthode + URL + en-têtes + corps)
- Le serveur renvoie une réponse HTTP (code de statut + en-têtes + corps)
- Ce schéma formalise ce qui était vu au Chapitre 1

Le principe stateless

- Chaque requête est traitée indépendamment
- Le serveur ne se souvient pas des requêtes précédentes
- Le client doit fournir tout le contexte nécessaire à chaque requête
- D'où l'existence des cookies de session, des jetons JWT
- Avantage : scalabilité horizontale — n'importe quel serveur peut traiter n'importe quelle requête

Anatomie d'une requête HTTP

- Une URL : quelle ressource ?
- Une méthode : quelle action ?
- Des en-têtes : quelles métadonnées ?
- Un corps optionnel : quelles données ?

Décomposer une URL

- Schéma : https
- Hôte : api.boutique.com (résolu par DNS)
- Port : 443 (omis si standard)
- Chemin : /produits/42
- Chaîne de requête : ?couleur=rouge&stock=oui
- Fragment : #avis — jamais transmis au serveur !

Les méthodes HTTP

- GET : récupérer — sûre, idempotente
- POST : créer — ni sûre, ni idempotente
- PUT : remplacer intégralement — idempotente
- PATCH : modifier partiellement — non idempotente
- DELETE : supprimer — idempotente

Sûreté et idempotence

- Sûre : ne modifie aucun état sur le serveur
- Idempotente : exécuter plusieurs fois produit le même effet qu'une seule fois
- POST n'est PAS idempotent : renvoyer deux fois crée deux ressources
- Conséquence pratique : ne jamais renvoyer automatiquement un POST après une erreur réseau

En-têtes et corps

- Host : nom de domaine visé
- User-Agent : identifie le logiciel client
- Content-Type : format du corps envoyé
- Authorization : identifiants / jetons
- Accept : formats de réponse acceptés
- Le corps transporte les données (JSON, formulaire…)

Anatomie d'une réponse HTTP

- Un code de statut : quel résultat ?
- Des en-têtes : quelles métadonnées ?
- Un corps optionnel : quelles données retournées ?

Les 5 familles de codes de statut

- 1xx — Informationnel : requête reçue, traitement en cours
- 2xx — Succès : 200 OK, 201 Created, 204 No Content
- 3xx — Redirection : 301 Moved Permanently
- 4xx — Erreur client : 400, 401, 403, 404
- 5xx — Erreur serveur : 500, 503

Codes essentiels à connaître

- 200 OK — succès générique
- 201 Created — ressource créée (après POST)
- 204 No Content — succès sans corps (après DELETE)
- 400 Bad Request — requête mal formée
- 401 Unauthorized / 403 Forbidden
- 404 Not Found — ressource inexistante
- 500 Internal Server Error / 503 Service Unavailable

HTTP, un protocole générique

- HTTP ne connaît rien du métier d'une entreprise
- C'est l'API, construite par-dessus, qui donne un sens métier
- Exemple : GET/POST/PUT/PATCH/DELETE sur /commandes/57
- Ceci annonce déjà le style REST, formalisé au Chapitre 4

Personnalisation par l'API

- En-têtes personnalisés (ex. X-Api-Key)
- Structure précise du corps imposée par la documentation
- Codes de statut combinés à un corps d'erreur détaillé

Exercices de la séance

- Décomposer une URL complète
- Identifier la sûreté et l'idempotence de chaque méthode
- Associer un code de statut à un scénario
- Écrire une requête HTTP complète

Synthèse

- Un protocole est un ensemble de règles partagées
- HTTP est le protocole applicatif dominant du web, sans état
- Une requête et une réponse ont chacune une anatomie précise
- Les méthodes se caractérisent par sûreté et idempotence
- HTTP est générique ; l'API lui donne un sens métier

Prochain chapitre

- Chapitre 3 : Types d'API et formats de données
- Que transporte le corps de ces messages ?
- Comment client et serveur s'accordent-ils sur le format ?